Robust Mini Linear Servo Actuator ——

mightyZAP ARDUINO API Manual

INDEX

1 Use Arduino Library

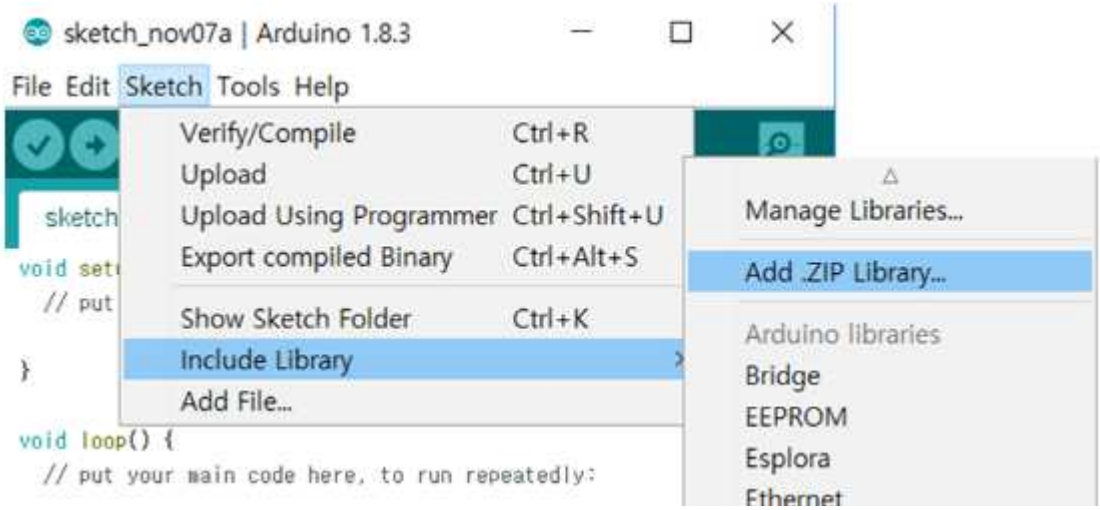
Arduino API can be downloaded from our website at http://www.irrobot.com/ (go to Digital Archive).
User may control servo actuatorconveniently usingArduinoAPI on Arduino IDE.
Arduino API is based on Arduino Leonardo and Uno. Users who use other Arduino boards, please amend API properly.

Follow below steps to use mightyZAP API.

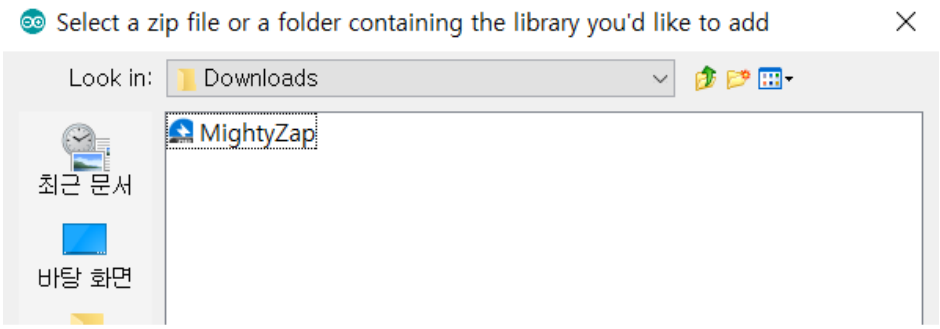
1. Add mightyZAP Library on Arduino IDE
2. Load mightyZAPExample
3. ProgramUpload

1.1 Add Library

1. Download Arduino library from www.irrobot.com– Digital Archive
2. In the menu, select [Sketch] – [Include Library] – [Add.ZIP Library…]



3. Select “MightyZAP.Zip” (File name is subject to be changed.)



1.2 ExampleLoading

1. Run Arduino IDE
2. [File] - [Example] - [MIGHTYZAP]– Select desiredExample

1.3 ProgramUpload

Select/Upload proper example according to Arduino board as below.

For Arduino UNO

[File] - [Example] - [MIGHTYZAP]– [UNO]- [MightyZap_Information]

For Arduino Leonardo

[File] - [Example] - [MIGHTYZAP] – [LEO]-[MightyZap_Information]

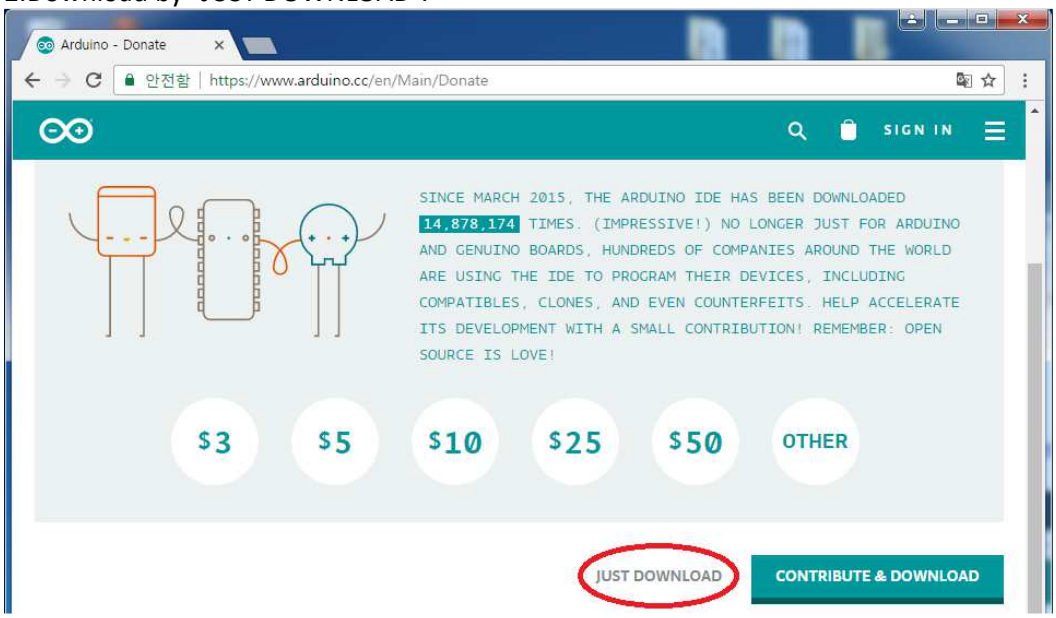
[Arduino PC Installation]

IR-STS01, Arduino Servo Tester Shield can be used with Arduino Uno or Arduino Leonardo. User needs to set Arduino development environment as below for program operation..

1. Install Arduino IDE

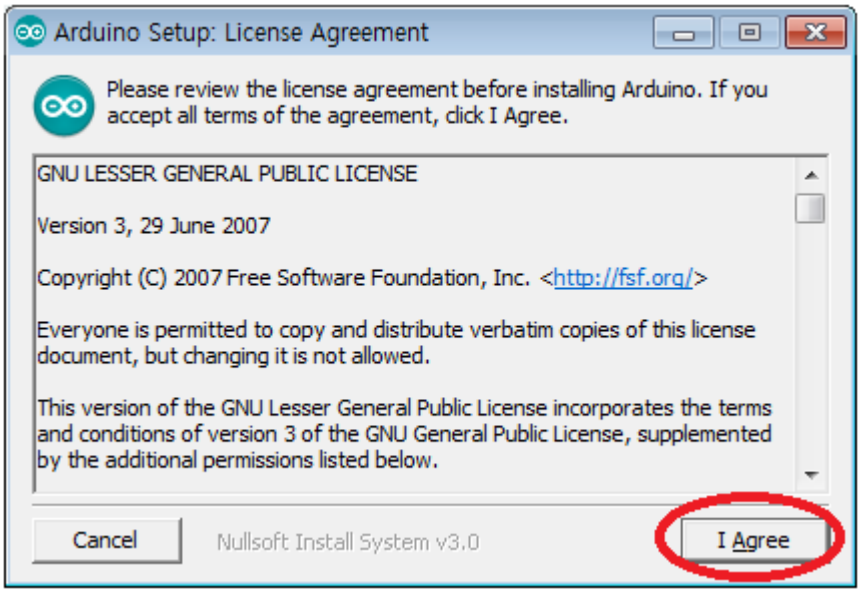
1. Select Window installer from https://www.arduino.cc/en/main/software

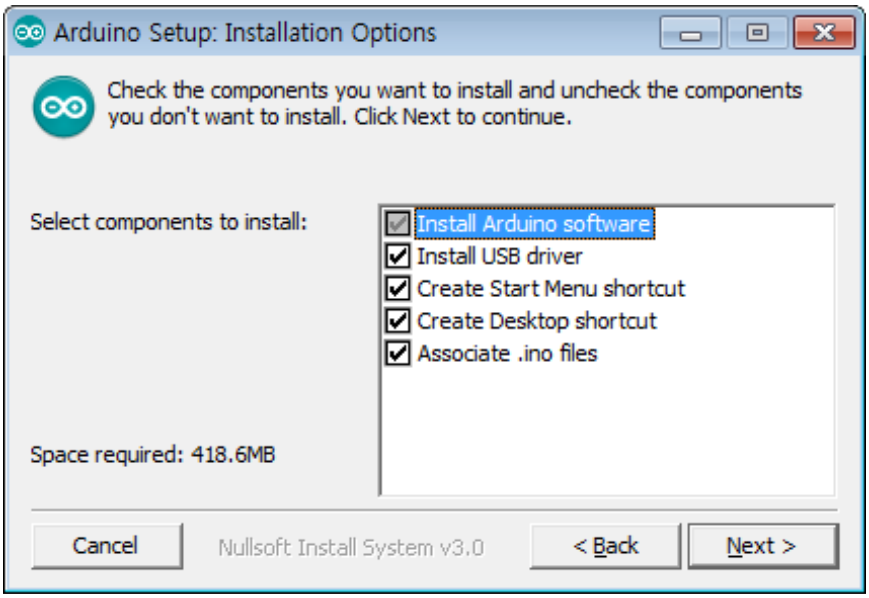
2.Download by “JUST DOWNLOAD”.

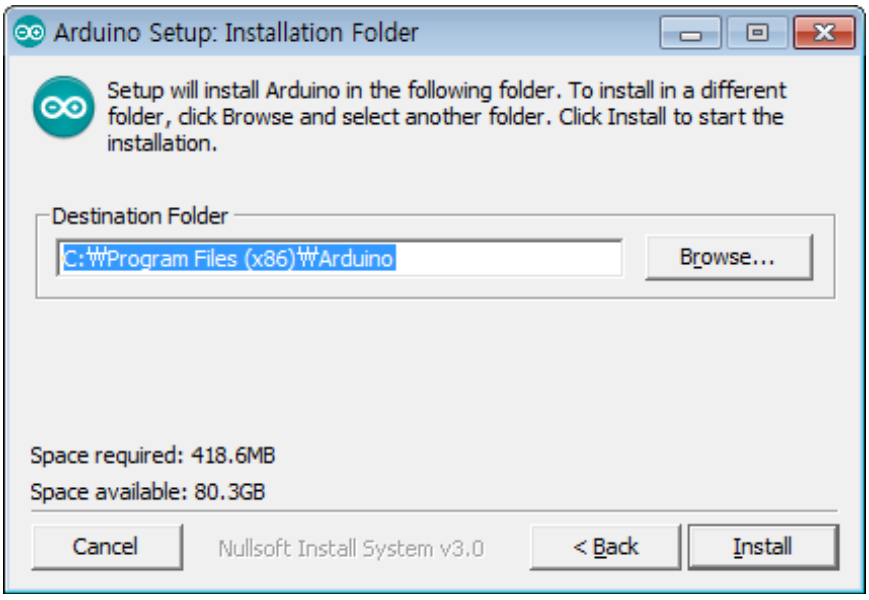


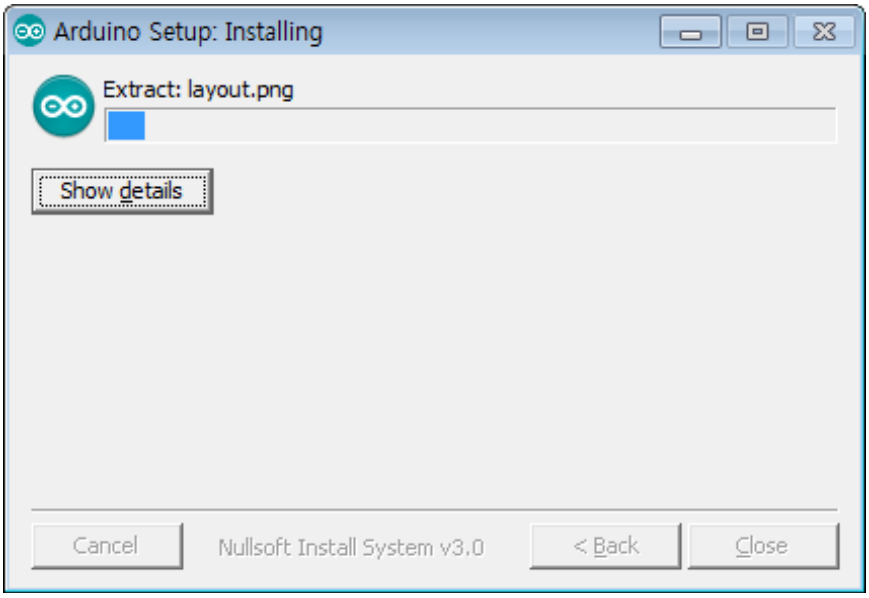
3. Run arduino-1.8.2.exe after download.

4. Install software as following.

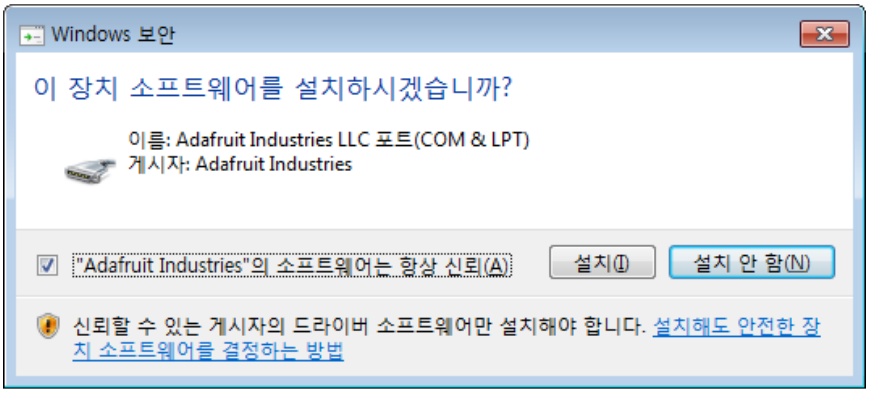




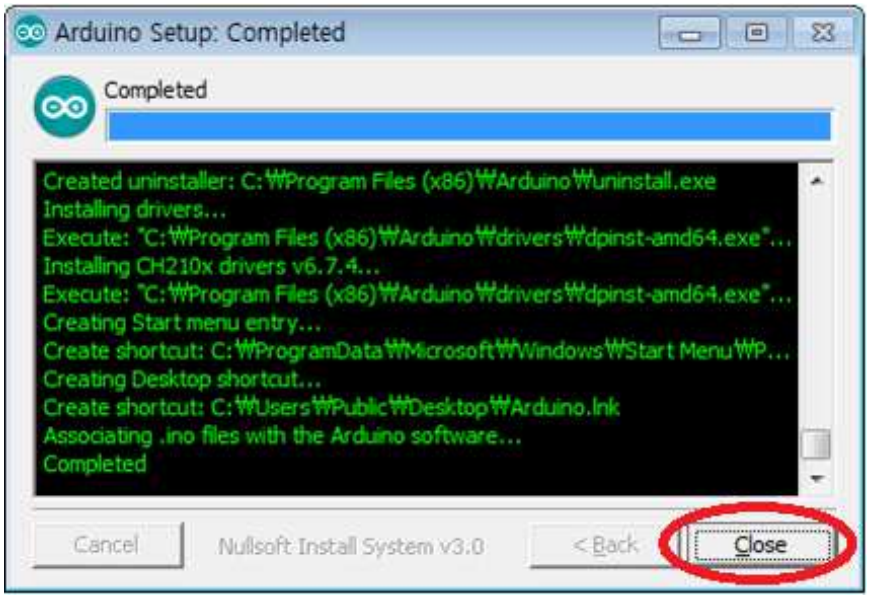




5. Install driver when Windows asks it.



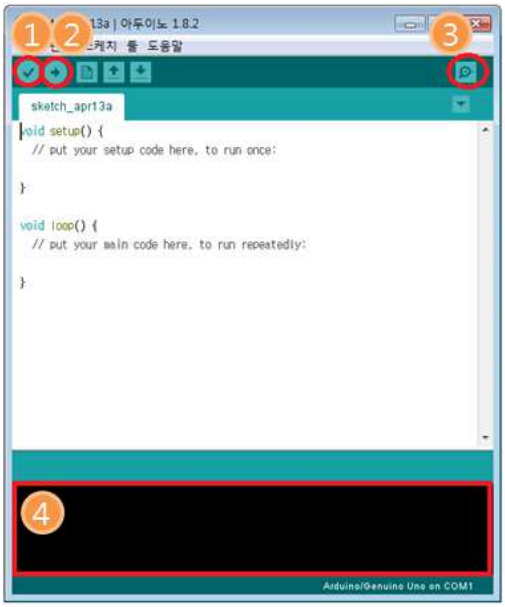
6. Click “Close” after installation.



7. Run “Arduino” on your Windows wallpaper.

2. Arduino IDE Structure

Basic composition of Arduino IDEis as below.



1 **Compile button**

2 **Upload button**

3 **Serial monitor button**

4 **Message area**

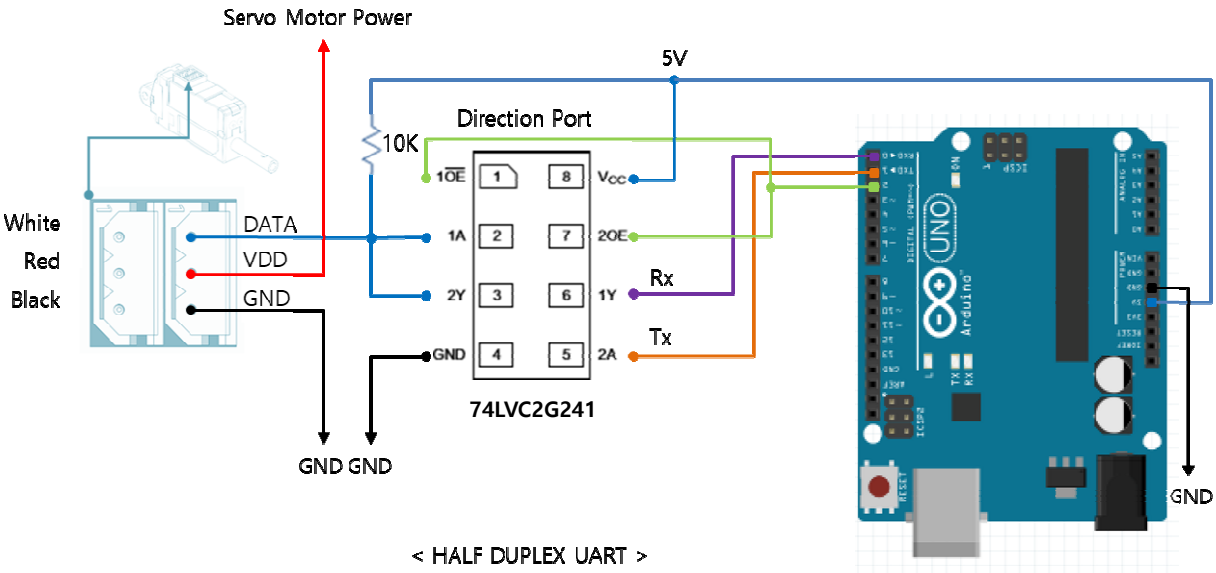
1. Compile button :Program will be compiled. (Compile result to be shown at area#4)
2. Upload button :Upload to Arduino at the same time of compile (In case of compile error or Arduino&usbconnection error occur, it will be shown at area#4.)
3. Serial monitor button :When PC is connected with Arduino via USB, Arduino is able to send message to PC when its operation. If program is written by Serial.write() orSerial.print() functions, user is able to check message on Serial monitor.
4. Message area :Various Error message, complie, upload result will be shown in this area.

3. Servo Connection (TTL/RS-485)

mightyZAP supports both data communication(Half Duplux TTL) as well as simple pulse(R/C hobby PWM) control. For the control under data communication, UART signal of main board should be converted into Half Duplex Type signal. Conversion circuit will be as below.(For users who use IR-STS01 Servo Tester Shield, does not need to build this TTL circuit as IR-STS01 already includes the circuitry.)

User who prefersPWM connection, see chapter 4.

3.1. TTL (3Pin connector)



As shown above, the data direction of TX and RX for TTL level will be determined according to the level of DIRECTION_PORT as below.

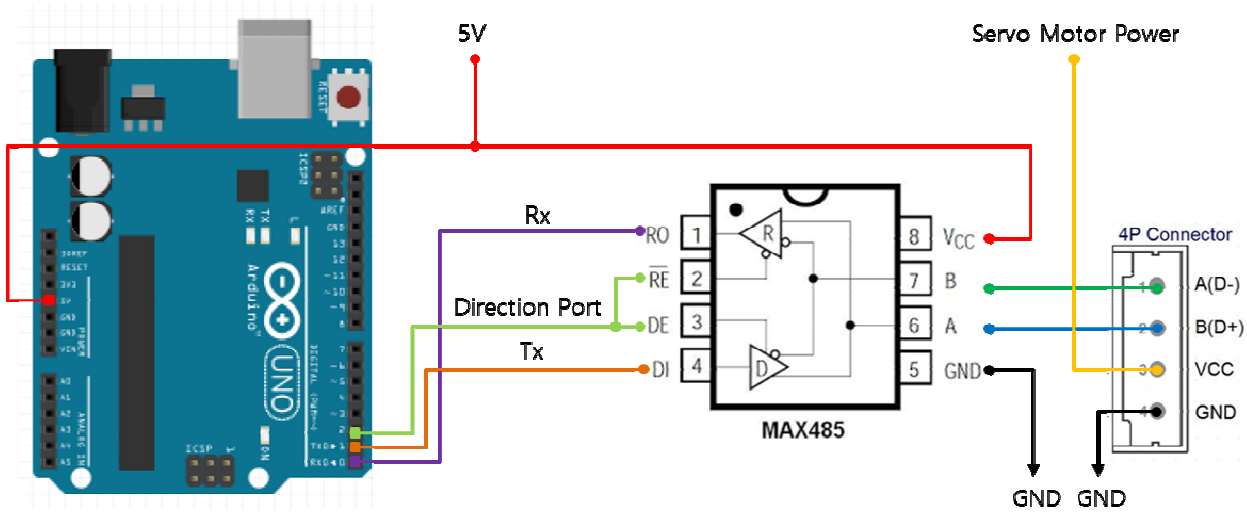
- In case of DIRECTION_PORT's Signal Level is LOW. : Data Signal to be input to RX.
- In case of DIRECTION_PORT's Signal level is HIGH : TX signal to be output to Data.

※ If user build the circuit as shown above, use "MightyZap m_zap(&serial, pin, Tx_Lvel)" command to connect communication with servo actuator. At this time, set "&Serial" as concerned serial of Arduino board, "pin" as a pin number of Direction port, and "TX_Level" as 1. (For above picutre, it will be "MightyZap m_zap(&serial, 2, 1)")

3.2. RS-485 (4pin connector)

Pin map and conversion circuit are as below.

PIN NUMBER(COLOR)	PIN NAME	FUNCTION(RS485)
1(Yellow)	D-	RS485 –
2(White)	D+	RS485 +
3(Red)	VCC	Power +
4(Black)	GND	Power -



※If the power is supplied from outside, it would be fine to connect to 485 D+, 485 D- only.

You can convert TX and RX mode by controlling DIRECTION_PORT pin in above circuit.

- In case of DIRECTION_PORT's Signal Level is LOW. : Data Signal to be input to RX.
- In case of DIRECTION_PORT's Signal level is HIGH : TX signal to be output to Data.

※ If user build the circuit as shown above, use "MightyZap m_zap(&serial, pin, Tx_Lvel)" command to connect communication with servo actuator. At this time, set "&Serial" as concerned serial of Arduino board, "pin" as a pin number of Direction port, and "TX_Level" as 1. (For above picutre, it will be "MightyZap m_zap(&serial, 2, 1)")

2 Arduino API for mightyZAP

Control servo actuator on the Arduino IDE by data communication protocol (RS-485 or TTL). This can be used when users want to control the servo actuator by data communication. (Not PWM)

2.1. API Outline

Arduino API to control mightyZAP through Arduino board.
Please refer to the Mighty Zap user manual for detailed description of each parameter function.

“Memory” is non-volatile value, so the saved value is maintained even after the power is turned off, but the “Parameter” is volatile value, so the previous data is not saved when the power is turned off. Parameter value indicates the default value or current status when power is applied.

2.2 API List

- Force Control version actuator (model name starts with 12Lf) users can use all the following parameters.
- The parameter marked **[F/C version only]** applies only to the Force Control version actuator (model name starts with 12Lf) and does not apply to the Position control version actuator.
- Position Control version actuator (model name starts with L12 or D12) user can use all parameters except **[F/C version only]** parameters.

MightyZAP m_zap(&serial, pin) **/ Com. setting When using IR-STS01 MightyZAP Arduino Servo Sheild**

- Function: Communicationportsetting for servo actuator
- Parameter
  - Serial : When using the Arduino Leonardo board, it communicates with the Servo actuator through the Serial port 1.
  - pin : Switch transmission/reception using GPIO 2. (default : 2)

MightyZAP m_zap(&serial, pin, Tx_Level) **/Com. setting when using Arduinoboard only w/o shield.**

- Function : Communicationportsetting for servo actuator
- Parameter
  - Serial : When using the Arduino Leonardo board, it communicates with the Servo actuator through the Serial port 1.
  - pin : Switch transmission/reception using the corresponding GPIO pin.
  - Tx_Level : RS485 Control levelsetting(High or Low)

m_zap.begin(int baudrate)/ communication speed setting

- Function : Communication speed setting
- Parameter
  - baudrate : communicationspeedsetting with servo actuator

Data	Baudrate
16	115200
32	57600
64	19200
128	9600

m_zap.getModelNumber(int ID_NUM) / Parameter Read

- Function : Model number check of connected Servo actuator
- Parameter - ID_NUM : Servo actuator ID

m_zap.Virsion(int ID_NUM)/ Parameter Read

- Function : Servo actuator firmware version check
- Parameter - ID_NUM : Servo actuator ID

m_zap.PresentTemperature(int ID_NUM)/ Parameter Read

- Function : Present temperature check of Servo actuator (just for reference. Not exact info)
- Parameter - ID_NUM : Servo actuator ID

m_zap.LED(int ID_NUM, int Color)/ Parameter Write (Volatile)

- Function : Servo actuator LED setting
- Parameter
  - ID_NUM : Servo actuator ID
  - Color : Servo actuator LED Color

Data	Description
RED	RED ON
GREEN	GREEN ON
BLUE	BLUE ON [For Position Control lineup Only]

m_zap.ServoID(int ID_NUM, int ID_SET)/ Memory Write (Non-Volatile)

- Function : Servo actuator ID setting
- Parameter
  - ID_NUM : Servo actuator current ID
  - ID_SET : Servo actuator New ID

m_zap.GoalPosition(int ID_NUM, int postion)/ Parameter Write (Volatile)

- Function : Servo actuator Goal Positionsetting
- Parameter
  - ID_NUM : Servo actuator ID
  - Position : Goal position control between 0~4095 (defualt : 0~4095)

m_zap.GoalSpeed(int ID_NUM, int speed) / MemoryWrite (Non-Volatile) [FC Version Only]

- Function : moving speedvaluesetting of Servo actuator
- Parameter
  - ID_NUM : Servo actuator ID
  - Speed: Set actuator speed according to the value - 0, and between 1 ~ 1023.
  - 1~1023 : speed to be increased according to the value.
  - 0 : Max speed (same as speed data 1023)

m_zap.GoalCurrent(int ID_NUM, int current) / MemeoryWrite (Non-Volatile) [FC Version Only]

- Function : Max current limit setting of Servo actuator
- Parameter
  - ID_NUM : Servo actuator ID
  - current : Set actuator max current limit between 1 ~ 1600.
  - Value unit is mA and include tolerance (see data sheet)
  - The smaller the value, the weaker force and the slower speed.

m_zap.presentPosition(int ID_NUM)/ Parameter Read

- Function : Read present position of Servo actuator
- Parameter
  - ID_NUM : Servo actuator ID

m_zap.moving(int ID_NUM) / Parameter Read

- Function : Read moving status of Servo actuator
- Parameter
  - ID_NUM : Servo actuator ID

m_zap.ShortStrokeLimit(int ID_NUM, int position) / Memeory Write (Non-Volatile)

- Function : Set Short stroke limit(retracting direction) of Servo actuator
- Parameter
  - ID_NUM : Servo actuator ID
  - position: Set Short stroke limit(retracting direction) (0~4095)

m_zap.LongStrokeLimit(int ID_NUM, int position) / Memeory Write (Non-Volatile)

- Function : Set long stroke limit(extending direction) of Servo actuator
- Parameter
  - ID_NUM : Servo actuator ID
  - position: Set long stroke limit(extending direction) (0~4095)

m_zap.Acceleration(int ID_NUM, int acceleration) / Memeory Write (Non-Volatile) [FC Version Only]

- Function : Set acceleration rate of Servo actuator
- Parameter
  - ID_NUM : Servo actuator ID
  - acceleration : Set acceleration rate of Servo actuator(0~255)
    ※ The higher the value, the faster the acceleration.The lower the value, the smoother start.

m_zap.Deceleration(int ID_NUM, int deceleration) / Memeory Write (Non-Volatile) [FC Version Only]

- Function : Set deceleration rate of Servo actuator
- Parameter
  - ID_NUM : Servo actuator ID
  - deceleration : Set deceleration rate of Servo actuator(0~255)
    - ※ The lower the value, the smoother stop.
    - ※ If it is set too large, overshoot and minute vibration may occur when reaching the final position.
    - ※ If overshoot occurs while changing the basic settings of the servo motor such as Speed and GoalCurrent, you can reduce the deceleration rate.

m_zap.forceEnable(int ID_NUM, int value) / Parameter Write (Volatile)

- Function : Force On /Off of Servo actuator
- Parameter
  - ID_NUM : Servo actuator ID
  - value : Force On /Off of DC motor

  When Force off, the DC motor does not operate and it operates when GoalPosition command or Force On command is received.

  0 : Force Off
  1 : Force On
    - ※ Even if it is Forced Off, communication will still work.
    - ※ If the next position command is given in the Force Off state, the Force is automatically turned on and the position moves.

What is the Force Off?

The actuator lineup having rated load 35N or more, have a characteristic to keep the position due to the mechanical design characteristics without motor power.
Therefore, if the servo actuator in the facility needs to maintain a certain position for considerable time, you can extend the life of the actuator by shutting off the motor power with the Force Off command. In this case, the communication is still maintained, and only the motor is powered off. When the position movement command is given again, the force is automatically turned on and the next command is executed.

m_zap.readbyte(int ID_NUM, int Addr)

- Function : Read data directly from the address to read 1 byte data of Memory/Parameter in Data map.
- Parameter
  - ID_NUM : Servo actuator ID
  - Addr : Parameter address to read data

m_zap.readint(int ID_NUM, int Addr)

- Function : : Read data directly from the address to read 2 byte data of Memory/Parameter in Data map.
- Parameter
  - ID_NUM : Servo actuator ID
  - Addr : Parameter address to read data

m_zap.writebyte(int ID_NUM, int Addr, int Data)

- Function : Input data directly to the address to input 1 byte data of Memory/Parameter in Data map.
- Parameter
  - ID_NUM : Servo actuator ID
  - Addr : Parameter address to input data
  - Data : Inpt Data(0~255)

m_zap.writeint(int ID_NUM, int Addr, int Data)

- Function: Input data directly to the address to input 2 byte data of Memory/Parameter in Data map.
- Parameter
  - ID_NUM : Servo actuator ID
  - Addr : Parameter address to input data
  - Data : Input Data(0~4096)

3 ServoControlExample by Arduino IDE

Control servo actuator using data communication protocol (RS-485 or TTL) on Arduino IDE.

3.1 Outline

Arduino example is for serial data communication (RS-485 or TTL) only and it does NOT support PWM communication.

Arduino API is based on Arduino Leonardo and Uno. Users who use other Arduino boards, please amend API properly.

Refer to separate “mightyZAP user manual for detailed explanation about parameter function.

Use IR-STS01, Arduino Servo Tester Shield for more convenient connection and use.

3.2. Example- Information Read (RS-485/TTL)

Example to read basic information such as ID, firmware version number using mightyZAP API

Select [Example] - [MightyZAP] - [UNO OR LEO]–[MightyZAP_Information]

MightyZap_Information §

```
#include <MightyZap.h>

#define ID_NUM 0

MightyZap m_zap(&Serial1, 2);


void setup() {
  Serial.begin(9600);
  m_zap.begin(32);
  while (! Serial);
}

void loop() {
  if(Serial.available())  {
    char ch = Serial.read();
    if(ch=='d')    {
      Serial.print("Model Number        : ");  Serial.println((unsigned int)m_zap.getModelNumber(ID_NUM));
      Serial.print("Firmware Version    : ");  Serial.println(m_zap.Version(ID_NUM)*0.1);
      Serial.print("Present Temperature : ");  Serial.println(m_zap.presentTemperature(ID_NUM));
    }
  }
}
```

3.3. Example - LED

Setting LED color using LedON() command.
Select [Example] - [MightyZap] - [UNO OR LEO]– [MightyZAP_ LED]

Description

Control LED color using LedON() command.
Check LED color by RED, GREEN variable.

Alarm LED is highest priority regardless your LED setting.

3.4. Example - Servo ID

ID setting of servo actuator using ServoID() command.
Select [Example] - [MightyZap] - [UNO OR LEO]–[MightyZAP_ ServoID]

- Check LED color change by changing Servo ID.
- After changing ID of servo actuator, check ID change status by assign different LED color for changed ID.

- As all examples are using ID 0, so change ID to 0 again.

3.5. Example - Goal Position

Example to control servo actuator position using goalPosition() command.
Select [Example] - [MightyZap] - [UNO OR LEO]–[[MightyZAP_ goalPosition]

- By 1sec interval, control position between 0 ~ 4095.
- Change position value in m_zap.goalPosition(ID_NUM,Position value) function and check position difference.

3.6. Example - Present Position

Example to check current position of servo actuator using presentPosition() command.
Select [Example] - [MightyZap] - [UNO OR LEO]–[MightyZAP_ presentPosition]

MightyZap_PresentPosition

```
#include <MightyZap.h>
#define ID_NUM 0

MightyZap m_zap(&Serial1, 2);

int Position;
int cPosition;
int Display =1;

void setup() {
  Serial.begin(9600);
  m_zap.begin(32);
  while (! Serial);
  m_zap.GoalSpeed(ID_NUM,50);
}
void loop() {
  if(Display == 1){
    Serial.print("*New Position[0~4095] : ");
    Display = 0;
  }
  if(Serial.available())  {
    Position = Serial.parseInt();
    Serial.println(Position);
    delay(200);

    m_zap.GoalPosition(ID_NUM,Position);
    delay(50);
    while(m_zap.Moving(ID_NUM)) {
      cPosition = m_zap.presentPosition(ID_NUM);
      Serial.print("  - Position : ");
      Serial.println(cPosition);
    }
    delay(200);
    cPosition = m_zap.presentPosition(ID_NUM);
    Serial.print("  - final Position : ");
    Serial.println(cPosition);
    Display = 1;
  }
}
```

Open Arduino serial monitor and input position value, then check motor moving

After changing position using m_zap.goalPosition() command, check real-time present position bym_zap.presentPosition() command.

Check actuator's moving status by m_zap.moving() function. (1 : Moving, 0: Moving Stop)

3.7. Example –GoalSpeed[FC Version Only]

Set goal speed of actuator by using GoalSpeed() command.
Select [Example] - [MightyZap] - [UNO OR LEO]–[MightyZAP_ GoalSpeed]

MightyZap_GoalSpeed §

```
#include <MightyZap.h>
#define ID_NUM 0

MightyZap m_zap(&Serial1, 2);
int Speed, Display=1,cPosition;

void setup() {
  Serial.begin(9600);
  m_zap.begin(32);
  while (! Serial);
}

void loop() {
  if(Display ==1) {
    m_zap.GoalPosition(ID_NUM,0);
    delay(500);
    while(m_zap.Moving(ID_NUM)){
      cPosition = m_zap.presentPosition(ID_NUM);
      Serial.print("  - Position : ");
      Serial.println(cPosition);
    }
    m_zap.GoalPosition(ID_NUM,4095);
    delay(100);
    while(m_zap.Moving(ID_NUM)){
      cPosition = m_zap.presentPosition(ID_NUM);
      Serial.print("  - Position : ");
      Serial.println(cPosition);
    }
    Serial.print("New Speed[0~1023] : ");
    Display=0;
  }
  if(Serial.available()) {
    Speed = Serial.parseInt();
    m_zap.GoalSpeed(ID_NUM,Speed);
    Serial.println(m_zap.GoalSpeed(ID_NUM));
    delay(500);
    Display=1;
  }
}
```

- Open Arduino serial monitor and change Goal speed to see speed change.
- After setting the goal speed value with the m_zap.GoalSpeed() command, check the speed change using m_zap.GoalPosition() command.
- Use “moving()” command to check actuator’s operation status, and monitor position value by presentPosition() command during motor operation.

3.8. Example –GoalCurrent[FC Version Only]

Set maximum current limit of actuator by using GoalCurrent() command.
Select [Example] - [MightyZap] - [UNO OR LEO]–[MightyZAP_ GoalCurrent]

MightyZap_GoalCurrent§

```
#include <MightyZap.h>
#define ID_NUM 0

MightyZap m_zap(&Serial1, 2);
int Current, Display=1,cPosition;

void setup() {
  Serial.begin(9600);
  m_zap.begin(32);
  while (! Serial);
}

void loop() {
  if(Display ==1) {
    m_zap.GoalPosition(ID_NUM,0);
    delay(100);
    while(m_zap.Moving(ID_NUM)){
      cPosition = m_zap.presentPosition(ID_NUM);
      Serial.print("  - Position : ");
      Serial.println(cPosition);
    }
    m_zap.GoalPosition(ID_NUM,4095);
    delay(100);
    while(m_zap.Moving(ID_NUM)){
      cPosition = m_zap.presentPosition(ID_NUM);
      Serial.print("  - Position : ");
      Serial.println(cPosition);
    }
    Serial.print("New Current[0~1023] : ");
    Display=0;
  }
  if(Serial.available()) {
    Current = Serial.parseInt();
    m_zap.GoalCurrent(ID_NUM,Current);
    Serial.println(m_zap.GoalCurrent(ID_NUM));
    delay(500);
    Display=1;
  }
}
```

- Change goal current value and check current by goal position command.
- Set max current limit by M_zap.GoalCurrent() and then, check current difference by M_zap.GoalPosition() command.
- The force and speed of actuator will be changed according to GoalCurrent() value.
- Also, by moving()and presentPosition() command, monitor position value.

3.9.Example - Stroke Limit

Example to set stroke limit(limit for position) using LongStrokeLimit() &ShortStrokeLimit() command.
Select [Example] - [MightyZap] - [UNO OR LEO]–[MightyZAP_ StrokeLimit]

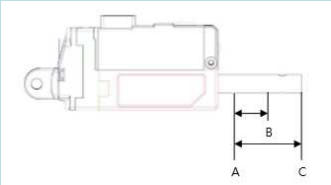
MightyZap_StrokeLimit§

```
#include <MightyZap.h>
#define ID_NUM 0

MightyZap m_zap(&Serial1, 2);

void setup() {
  m_zap.begin(32);
}

void loop() {
  m_zap.LongStrokeLimit(ID_NUM,3095);
  m_zap.GoalPosition(ID_NUM,4095);
  delay(5000);

  m_zap.ShortStrokeLimit(ID_NUM,100);
  m_zap.GoalPosition(ID_NUM,0);
  delay(5000);
}
```

m_zap.LongStrokeLimit()
- Setting max(long) position limit to control servo actuator position's limit.

m_zap.shortStrokeLimit()
- Setting min(short) position limit to control servo actuator position's limit.

Shot Stroke(A) : Retracing stroke
Long Stroke(C): Extending stroke

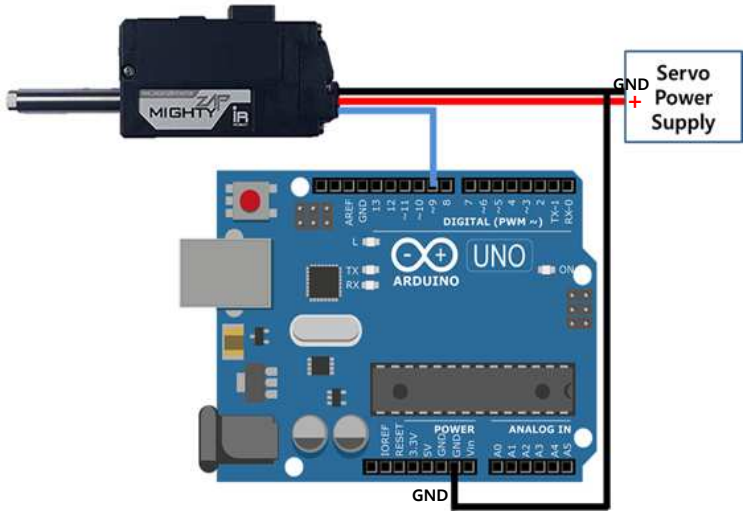


4 PWM Control Example via Arduino IDE

Control mightyZAP using Arduino IDE by PWM signal. L(D)-xxxPT-xx series supports both PWM and TTL. (PWM is not available with L-xxxF-xx series which support RS-485.)

4.1Outline

Proper circuitry needs to be made to control mightyZAP with Arduino as below. Please make sure correct input voltage should be applied according to your mightyZAP model. (7.4V or 12.1V).



3 pin wire consists of Power(Red), Ground(Black) and Signal(White)

- Power : Connect proper input power (7.4V or 12.1V) to your mightyZAP.
- GND : Connect GND between mightyZAP and Arduino board.
- Signal : Connect Signal to Pin#10.

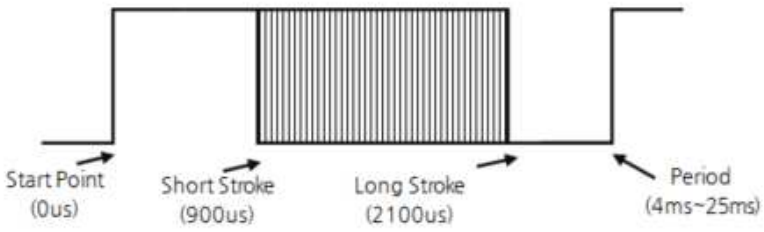
4.2 Position Control with PWM

Control position of mightyZAP using PWM library of Arduino.

BLE_Name_Setting §

```
#include <Servo.h>
Servo myservo;

void setup() {
  myservo.attach(10);
}

void loop() {
  myservo.writeMicroseconds(1200);
  delay(4000);

  myservo.writeMicroseconds(1800);
  delay(4000);
}
```

- #include <Servo.h> : Include header file to use Servo library.
- Servo myservo : Define the name of servo motor to control.
- Servo.attach(servoPin) : Assign the pin of the defined servo motor. (Pin#10 is assigned for example.)
- Servo.writeMicroseconds() : Put PWM pulse width in μs unit. (900 ~ 2100us) For example, Pulse width 1200 & 1800 has been applied here with 4000ms delay. 4000ms)
- Servo.write() : Conventional control method of rotary type R/C Servo motor. Put value between 0 ~ 180 to control position.



Rod Stroke	Pulse Mode
Short Stroke	900us
Half Stroke	1500us
Long Stroke (for 27mm strk)	2100us
Long Stroke (for 30mm strk)	2100us

[IR-STS01 - ArduinoServo Tester Shield]

Easier Arduino control with IR-STS01 Arduino Servo Tester Shield. Consist of Arduino Leonardo, plus our own Servo tester shield, user does not need complicated wiring and use it immediately

For more detailed information, please refer to the manual for IR-STS01.